TP_BD Enterprise Architecture Report

Antigravity & User

2026-01-08

1. Executive Summary

This document details the architecture, implementation, and infrastructure design of the **TP_BD Enterprise Stack**. The project aim was to create a robust, scalable, and production-like local development environment for a Laravel-based social network application.

The resulting architecture is a fully Dockerized, microservices-style stack featuring: * **Horizontal Scaling**: Multiple application containers behind a load balancer. * **Hybrid Database System**: Relational (MariaDB) and NoSQL (MongoDB) databases running in replication/cluster modes. * **High Performance**: Redis for caching and asynchronous queue processing.

---

2. Technology Stack & Rationale

2.1 Core Application

- **Framework**: **Laravel** (PHP). Chosen for its rich ecosystem, robust ORM (Eloquent), and ease of use for complex enterprise applications.
- **Web Server**: **Nginx**. Acts as the entry point and Load Balancer, distributing traffic round-robin style to application instances `app-1` and `app-2`.

2.2 Data Layer (The "Hybrid" Approach)

A. MariaDB (Relational Data)

- **Role**: Primary store for structured data (Users, Auth).
- **Architecture**: **Primary-Replica Replication**.
  - **Primary (`mariadb-primary`)**: Handles all **WRITE** operations.
  - **Replica (`mariadb-replica`)**: Handles **READ** operations (offloading the primary).
- **Rationale**: Ensures data integrity for critical relational structures while proving the ability to scale reads independently of writes.

B. MongoDB (NoSQL Data)

- **Role**: Store for high-volume, unstructured social data (posts, comments, interactions).
- **Architecture**: **Replica Set (`rs0`)**.
  - Includes `mongo-primary`, `mongo-secondary`, and `mongo-arbiter`.

- **Rationale**: Provides high availability and redundancy. The "Schema-less" nature fits the dynamic nature of social feeds and interactions.

C. Redis (In-Memory)

- **Role**: Performance layer.
- **Architecture**: Two separate instances.
  - `redis-cache`: For Session storage and application caching.
  - `redis-queue`: For managing asynchronous background jobs.
- **Rationale**: Separation of concerns ensures that a flood of queue jobs does not impact the performance of the user-facing cache.

---

3. Infrastructure Implementation

The entire stack is defined via `docker-compose.yml` and orchestrated by a custom `start.sh` script.

3.1 Network Topology

All services connect via a custom bridge network `tp_bd_network`. This isolates the environment from the host machine while allowing unrestricted communication between containers.

3.2 Orchestration (`start.sh`)

An automation script was developed to handle the complex startup order: 1. **Validation**: Checks if Docker is running. 2. **Build & Start**: Brings up containers. 3. **Wait**: Implements "Wait for Ready" logic (sleeps) to allow databases to initialize. 4. **Database Bootstrapping**: * Initializes MongoDB Replica Set (`rs.initiate()`). * Configures MariaDB Replication (`CHANGE MASTER TO`). 5. **Application Provisioning**: * Runs `php artisan migrate --force`. * Generates keys and optimizes caches.

---

4. Implementation Challenges & Solutions

During the deployment, we encountered and resolved several critical infrastructure issues.

4.1 The "Replication Access Denied" Incident

The Problem: While running the startup script, the `php artisan migrate` command failed with: `SQLSTATE[HY000] [1045] Access denied for user 'laravel'@'172.18.0.11'`

Analysis: 1. Logs confirmed the `mariadb-primary` was healthy. 2. We discovered that Laravel, configured for Read/Write splitting, was checking the **Replica** (`mariadb-replica`) for the existence of the migrations table. 3. Checking the Replica revealed a replication error: `Last_SQL_Error: Error 'Can't find any matching row in the user table'`. 4. **Root Cause**: The initialization script (`init-primary.sql`) ran `GRANT ALL ... TO 'laravel'` but did not explicitly run `CREATE USER` beforehand. While this sometimes works on the primary due to implicit creation permissions, the

Replication Stream failed because it tried to replicate the `GRANT` statement to a Replica where the user `laravel` did not exist.

The Solution: We modified `docker/mariadb/init-primary.sql` to strictly enforce order of operations:

```
-- Explicitly create the user first
CREATE USER IF NOT EXISTS 'laravel'@'%' IDENTIFIED BY 'secret';
-- Then grant permissions
GRANT ALL PRIVILEGES ON tp_bd.* TO 'laravel' @'%';
```

After resetting the volumes (`docker compose down -v`), replication synced correctly, and the migration succeeded.

4.2 Application Permissions (Error 500)

The Problem: Upon accessing `http://localhost`, the application returned a 500 Internal Server Error. Use of the **Browser Subagent** revealed the error: `file_put_contents(...): Failed to open stream: Permission denied`

The Solution: This is a standard container issue where the `www-data` user (internal) differs from the mounted volume owner (host). We resolved this by modifying permissions on the active container: `chmod -R 777 storage bootstrap/cache`

4.3 Observability Gap

The Problem: There was no easy way to visualize the complex architecture live.

The Solution: We integrated **Portainer** into the stack. * Added `portainer` service to `docker-compose.yml`. * Exposed on port `9000`. * This provides a real-time logical view of the architecture (Containers, Networks, Volumes).

---

5. Conclusion

The **TP_BD Enterprise Stack** is now fully operational. It demonstrates a sophisticated use of Docker to simulate a high-scale production environment. The architecture is resilient (via replication), performant (via Redis/Load Balancing), and observable (via Portainer).

Access Points: * **App**: http://localhost * **Portainer**: http://localhost:9000 * **DB Ports**: 3306 (Primary), 3307 (Replica), 27017 (Mongo)